Synergy '26 / StackSense Challenge

StackSense

Intelligent Tech Stack Recommendation Engine

A deterministic architecture workbench for evaluating project constraints.

Submission metadata pending: entrant name, college, Unstop email and optional repository URL.

10 November 2026

1. Abstract

StackSense converts a structured project scenario into an architecture recommendation, cost band, trade-offs and warnings. It runs offline in a browser or through a dependency-free Node CLI. The engine validates the shared schema, selects the deployment boundary and scores compatible backend options. Supporting services and cost are evaluated alongside the core stack. Generated JSON preserves the drivers, alternatives and planning assumptions used in the decision.

2. Problem understanding

The useful result is a stack a team can investigate within its constraints. The engine rejects explicit deployment conflicts and estimates budget fit before returning a stack. Delivery speed, language fit, workload and operating burden guide selection. Unknown regions and external data paths are surfaced for review. Provider products are design candidates, not compliance certifications.

3. System architecture

Browser form or CLI JSON > shared schema and validation > constraint checks > provider and region selection > scoring and capability rules > cost gate > decision and JSON export.

schema.js defines field types and ranges. engine.js applies defaults, validates, and generates the same result in both runtimes. app.js maps form controls to the JSON scenario and presents cost, conflicts and component details. cli.js supports individual and batch exports. The optional HTTP server provides the same engine over a local endpoint.

4. Input design and research

Project type and requested platforms drive architecture shape and client coverage. Name identifies the project and description provides context. User counts, concurrency, traffic pattern, workload and volume inform scale warnings, backend scoring, caching and storage. Team size, languages, experience and DevOps capacity inform framework fit and operational warnings. Budget and delivery horizon set the cost gate and delivery weighting.

Cloud, deployment model, residency and existing systems resolve the provider and region. Requirements cover identity, realtime, offline behavior, search, storage, payments, tenancy, ML, SEO and service targets. Compliance flags require control and contract review. Priorities steer scoring, search and cache selection, and resilience allowances.

Optional inputs cover throughput, recovery, sensitivity, portability, search size, concurrent editing, ledger model, offline writes and resumable uploads. They change sizing, safeguards or selected components. The shared schema rejects malformed sections, unknown fields and values, missing required inputs and invalid numeric ranges.

5. Knowledge base and decision logic

AWS, Azure and Google Cloud profiles bind hosting, database and object storage to the selected regional provider. Existing Microsoft identity influences provider selection only when the cloud choice is unrestricted. VM mode selects a provider VM and self-operated data services. Local mode uses local hosting, build scripts and telemetry. An unknown required region or incompatible air-gap dependency returns an infeasible result.

Backend score = language fit bonus + delivery rating times delivery weight + throughput rating times realtime load weight. A small or beginner team raises the language-fit weight. A short MVP raises the delivery weight. Performance and scalability priorities raise the realtime term. Ordinal ratings are judgment-based, and catalog order settles ties. The winning and runner-up scores are returned.

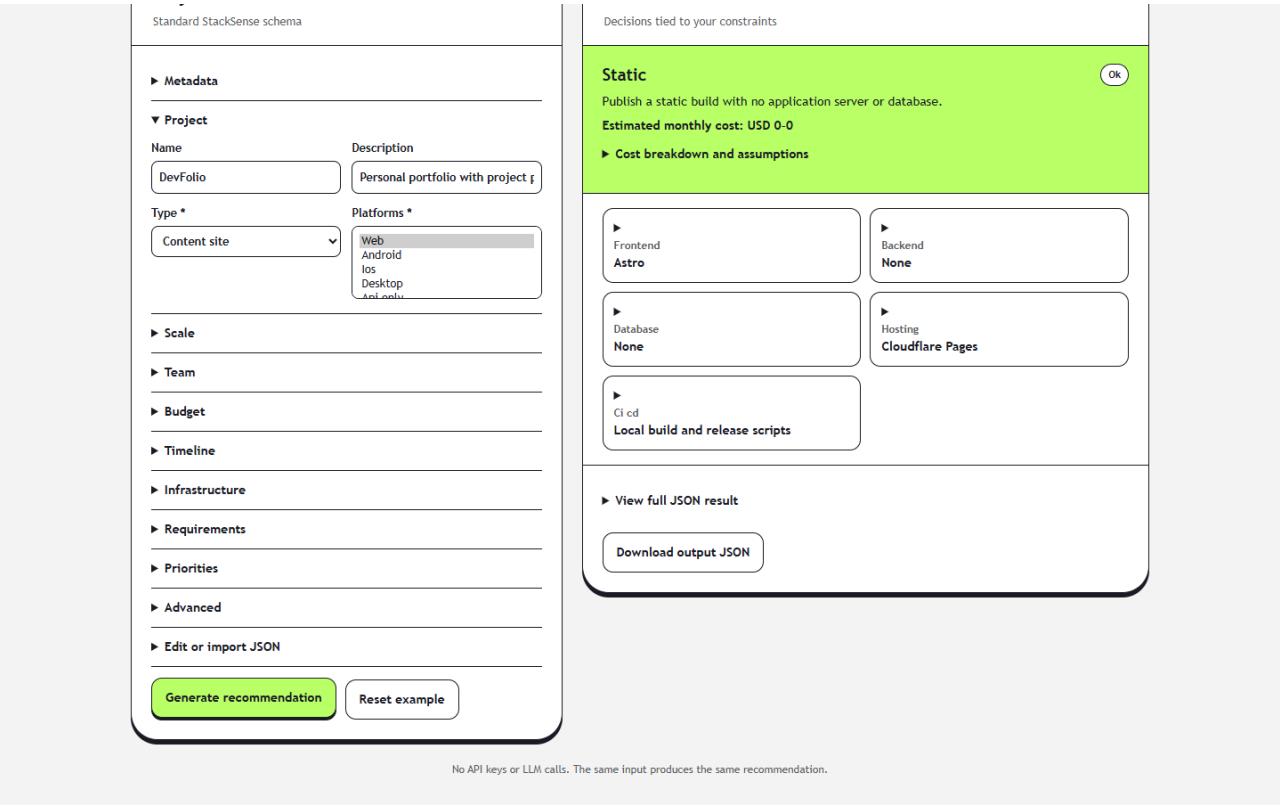
A cost band sums compute, database, storage, identity and applicable supporting workloads. App replica count comes from peak requests and a load-test target of 100 requests per second per replica. Database sizing uses explicit storage assumptions. A minimum above budget blocks the stack. An overlapping band retains the warning and offers a smaller pilot with spending allocations. Taxes, labor, transaction fees, SMS and accelerators are excluded from the base band.

The engine makes no model calls. No API key, network access or feature switch is needed to reproduce the output.

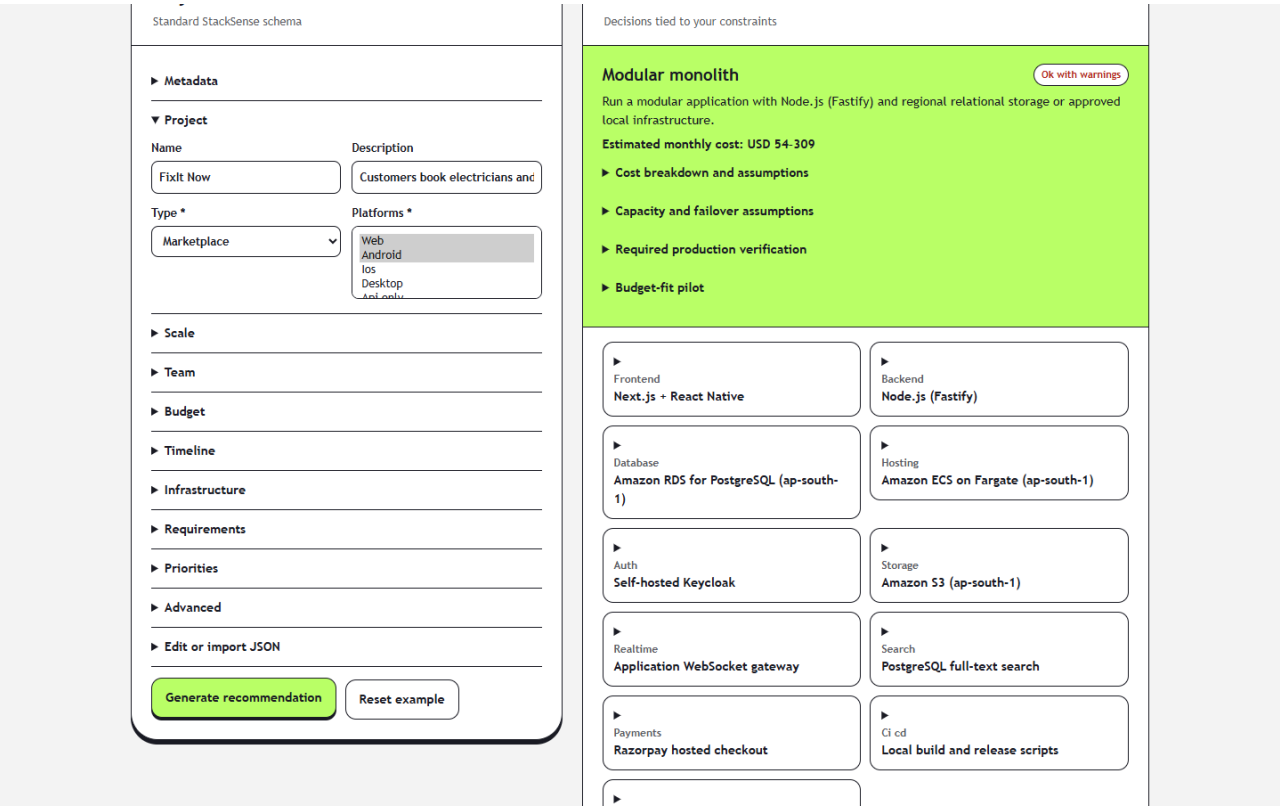
6. Sample recommendations

6. Public scenarios P1 and P2

P1 / Astro / Cloudflare Pages / USD 0-0 per month.

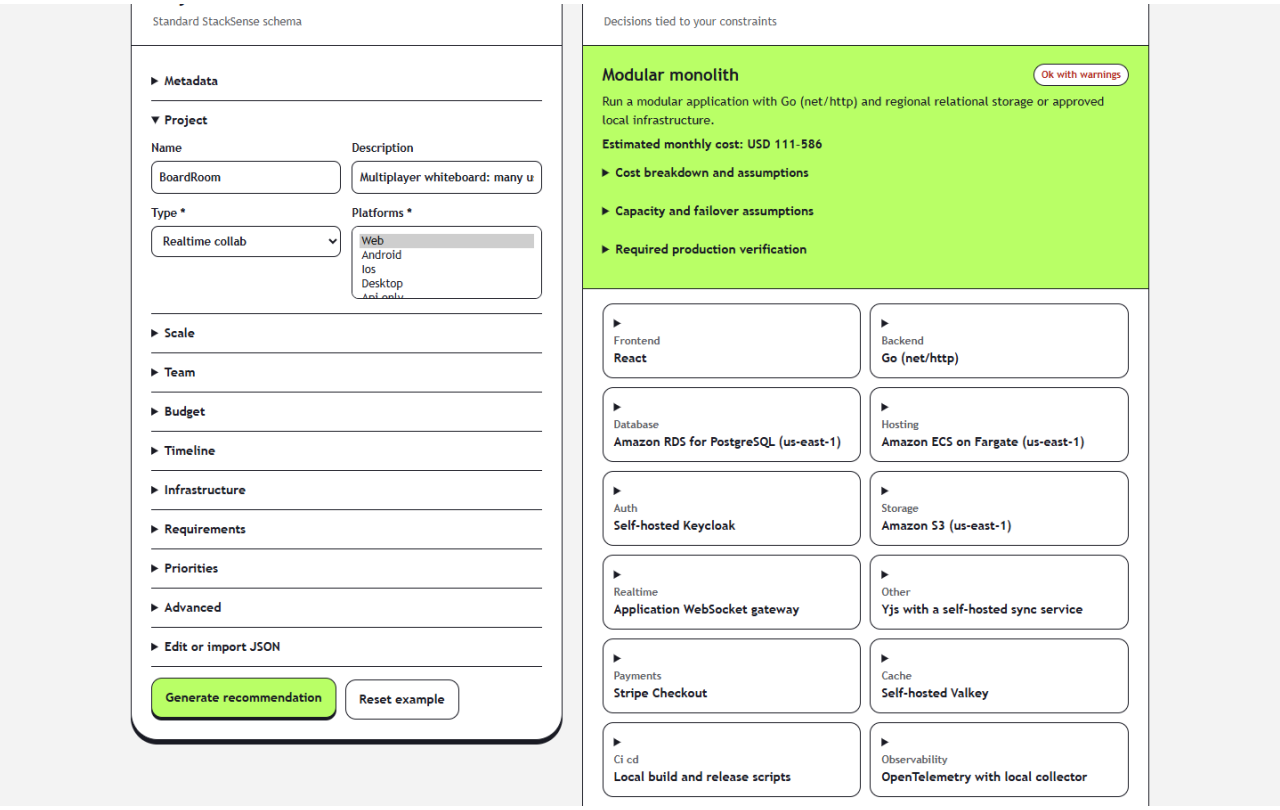


P2 / Next.js + React Native / Amazon ECS on Fargate (ap-south-1) / USD 54-309 per month.

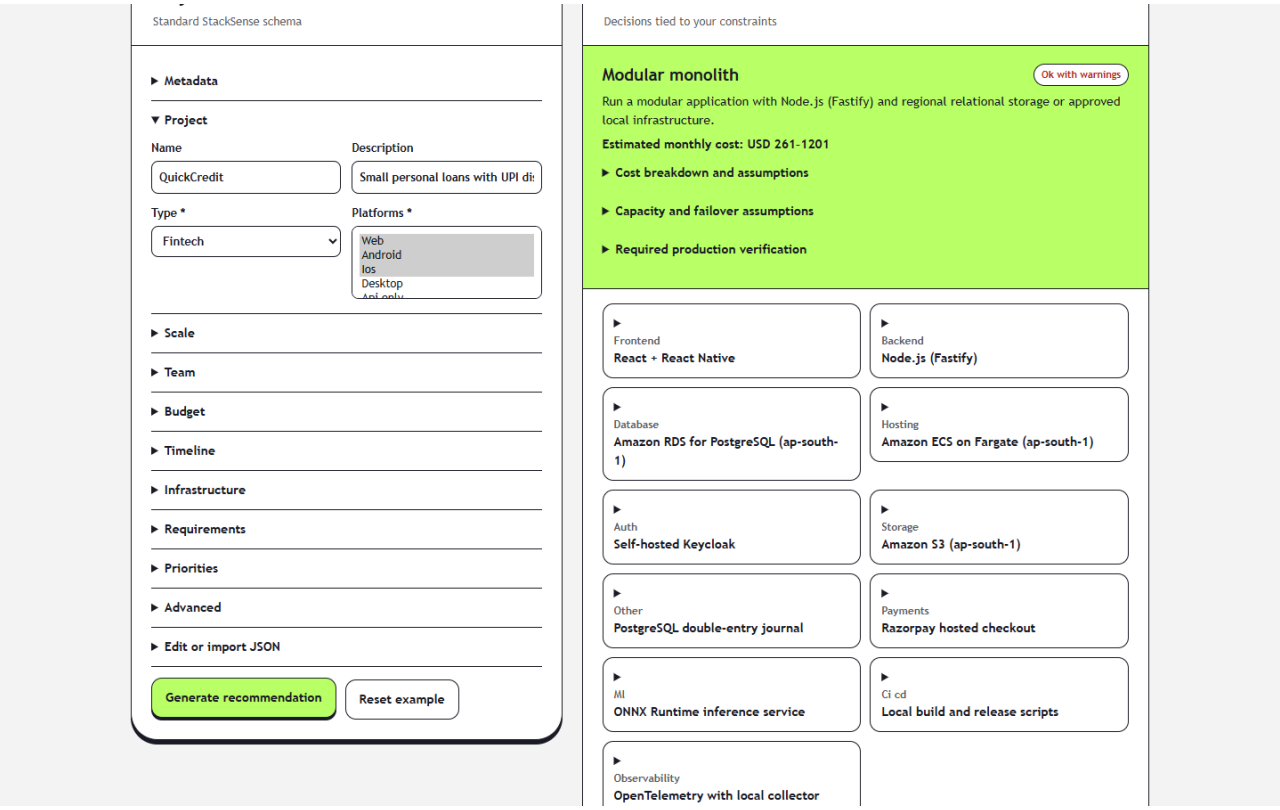


6. Public scenarios P3 and P4

P3 / React / Amazon ECS on Fargate (us-east-1) / USD 111-586 per month.

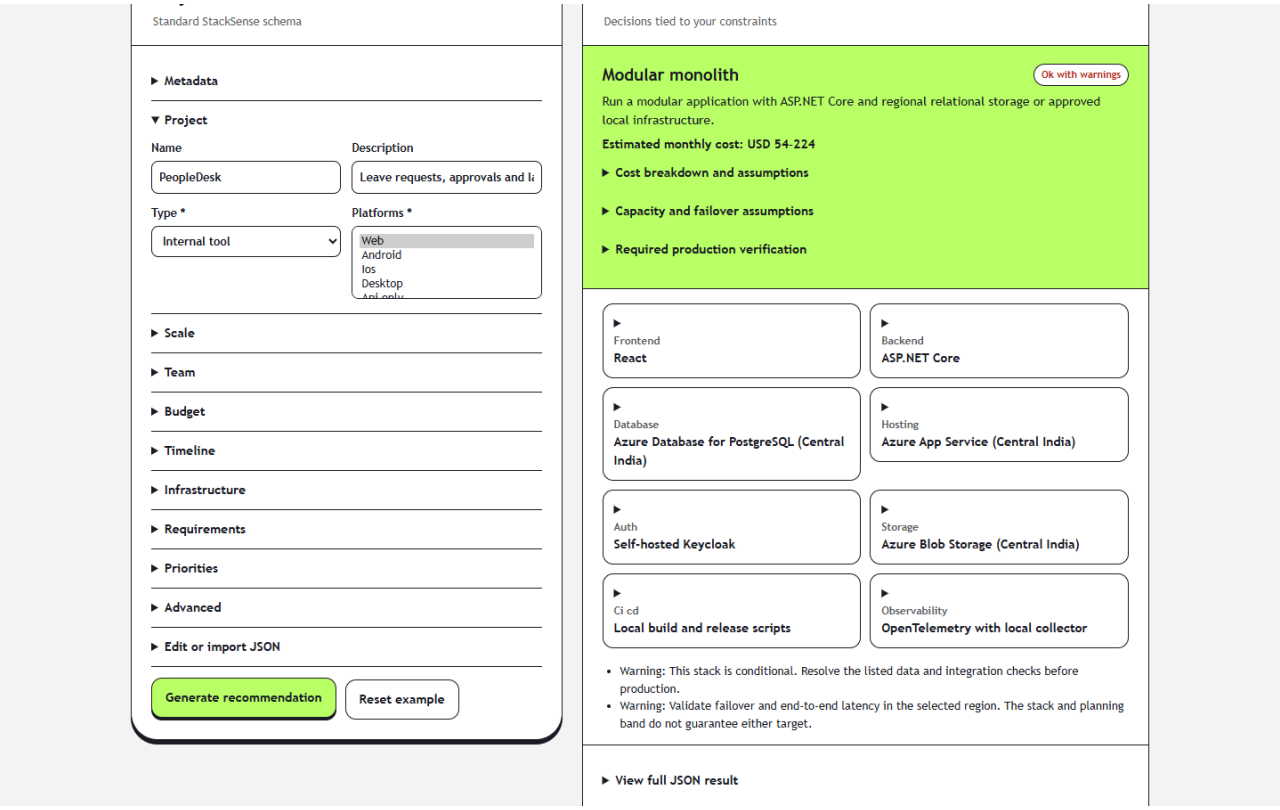


P4 / React + React Native / Amazon ECS on Fargate (ap-south-1) / USD 261-1201 per month.

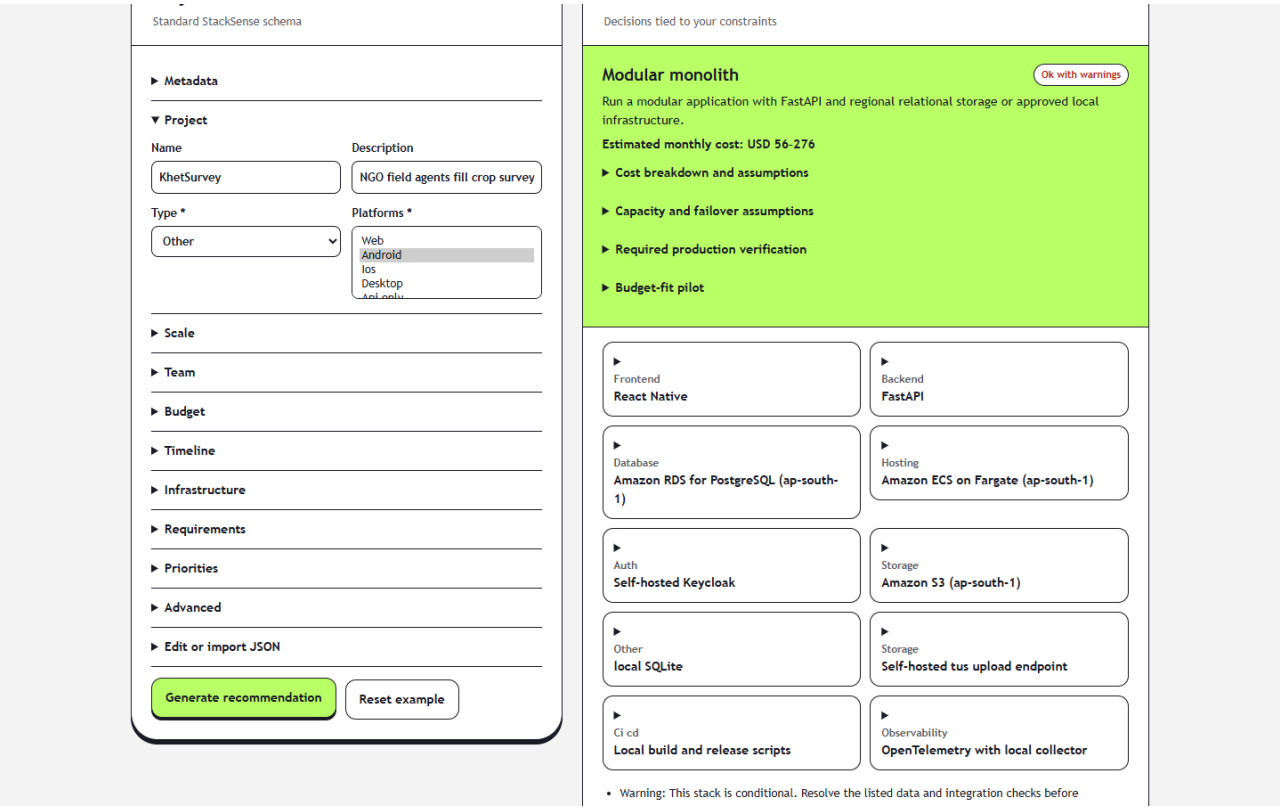


6. Public scenarios P5 and P6

P5 / React / Azure App Service (Central India) / USD 54-224 per month.



P6 / React Native / Amazon ECS on Fargate (ap-south-1) / USD 56-276 per month.



6. Decision walkthroughs

P2, FixIt Now. Next.js covers discoverable web pages and React Native covers Android. Fastify matches the declared JavaScript and TypeScript experience. Regional PostgreSQL supports transactions and search, avoiding a separate search service. Fargate and S3 stay in the selected Indian region. The app owns the WebSocket gateway. Keycloak is deployed in the same boundary, while the SMS and payment integrations need their own data-flow review. The upper band exceeds budget, so the result warns.

P5, PeopleDesk. React covers the web client. ASP.NET Core matches C# experience. Azure App Service, managed PostgreSQL and Blob Storage share the Azure deployment region. Keycloak can broker enterprise sign-in, subject to IdP configuration and data review. Validate any required SQL Server integration separately. Verify latency and backup retention with representative HR records.

P2 pilot. The monthly target is USD 130.77, with USD 19.23 unallocated. Start with one rollout cohort. Preserve realtime and payment flows. Gate new enrollment on observed session and transaction costs. Confirm regional pricing, metered usage and service targets before launch.

P6 pilot. The monthly target is USD 91.20, with USD 8.80 unallocated. Start with one field cohort. Preserve offline capture and queued uploads. Defer onboarding when sync backlog or metered usage exceeds the pilot envelope. Confirm regional pricing, metered usage and service targets before launch.

7. Edge-case handling

Missing fields and invalid values: reject before selection and return per-field errors.

Contradictions: return infeasible for incompatible platforms, deployment and identity choices or a modeled minimum above budget.

Over-engineering: P1 stays static and needs no API or database. Search can reuse the main database.

Regulatory constraints: select a supported region and flag external processing paths, contracts and configuration.

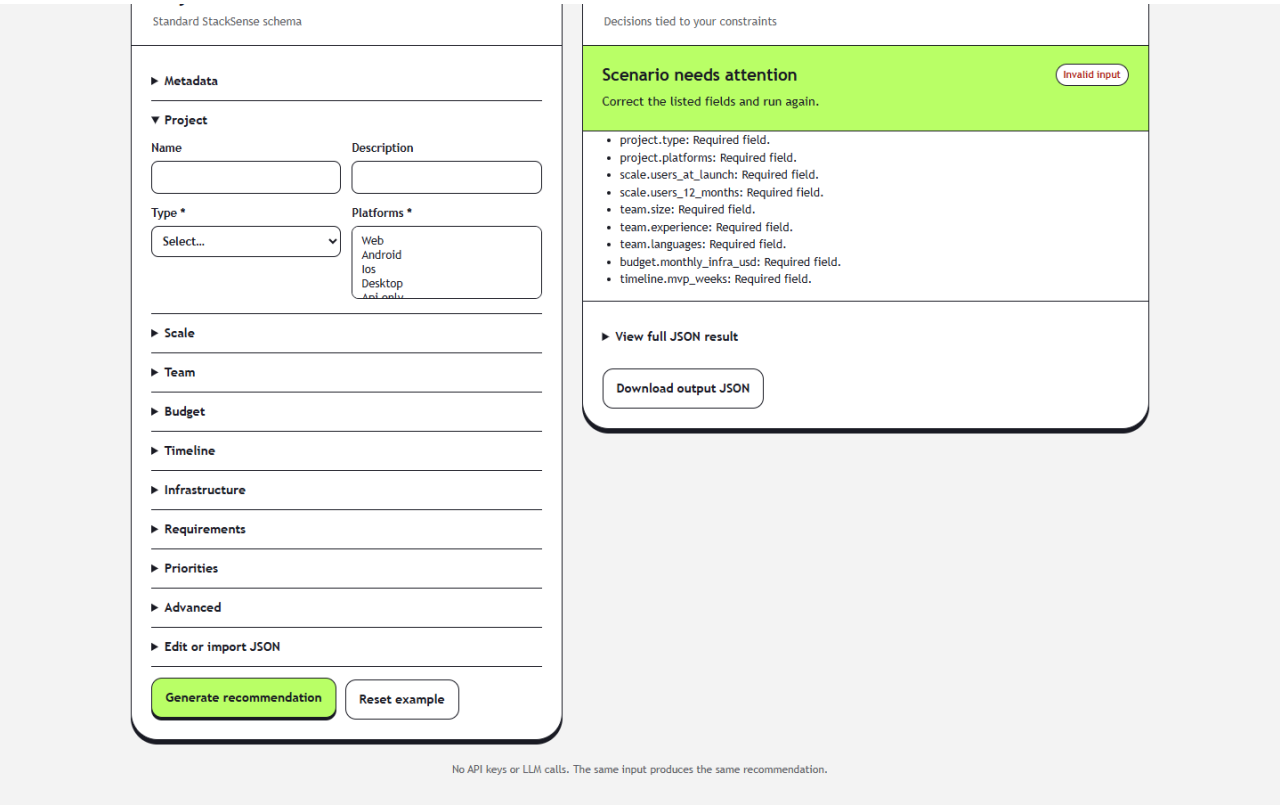
Existing systems and skill mismatch: use identity hints only when permitted and score declared skills explicitly.

Restricted deployment: serve and build inside the boundary with local tooling.

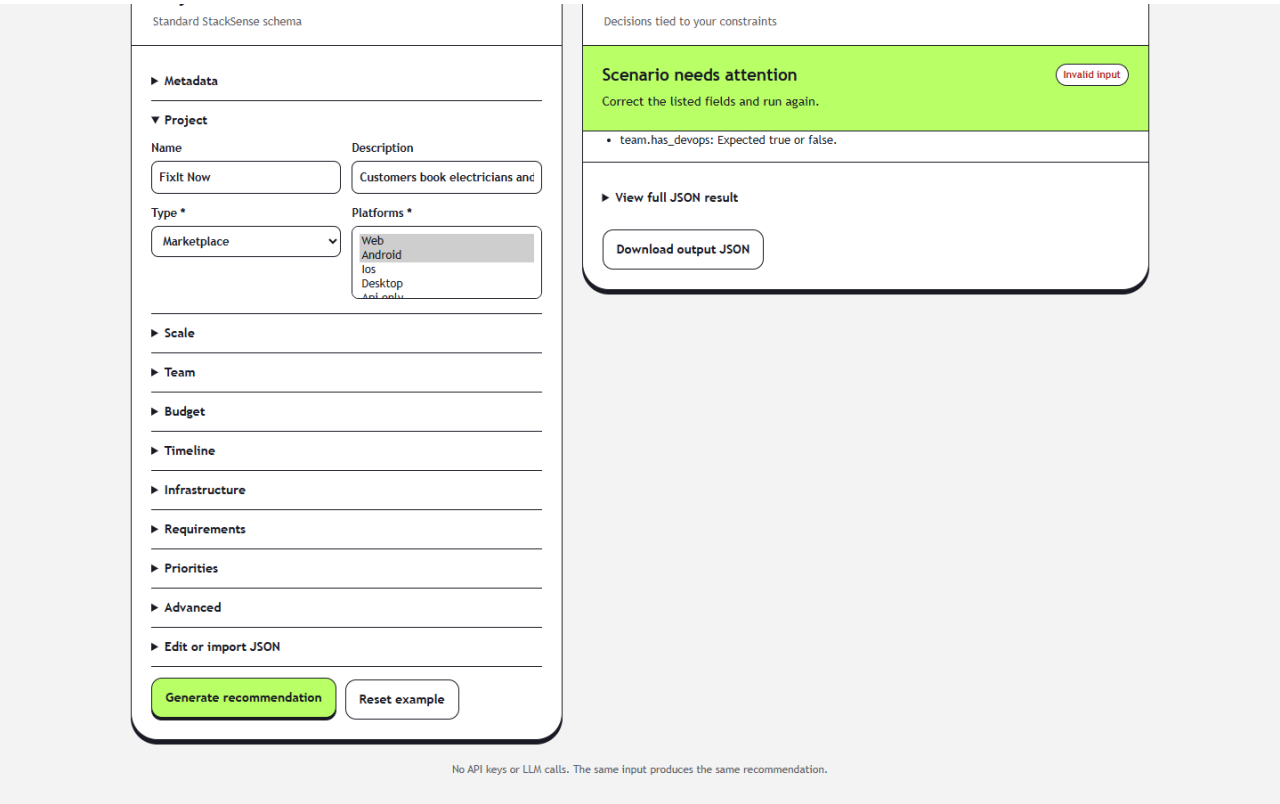
Retired technology: the catalog requires manual lifecycle review. Existing-system end-of-life detection is not automated.

7. Captured edge cases

Missing required fields

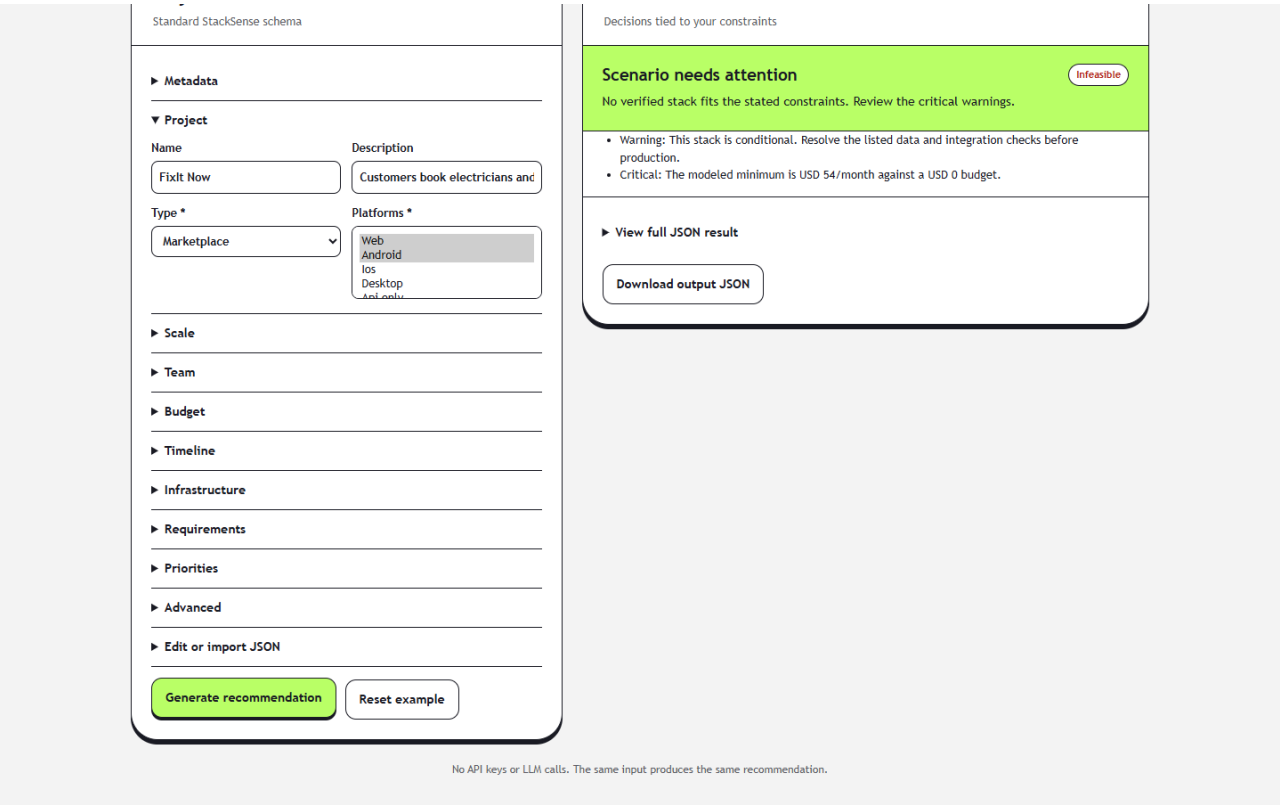


Invalid boolean field

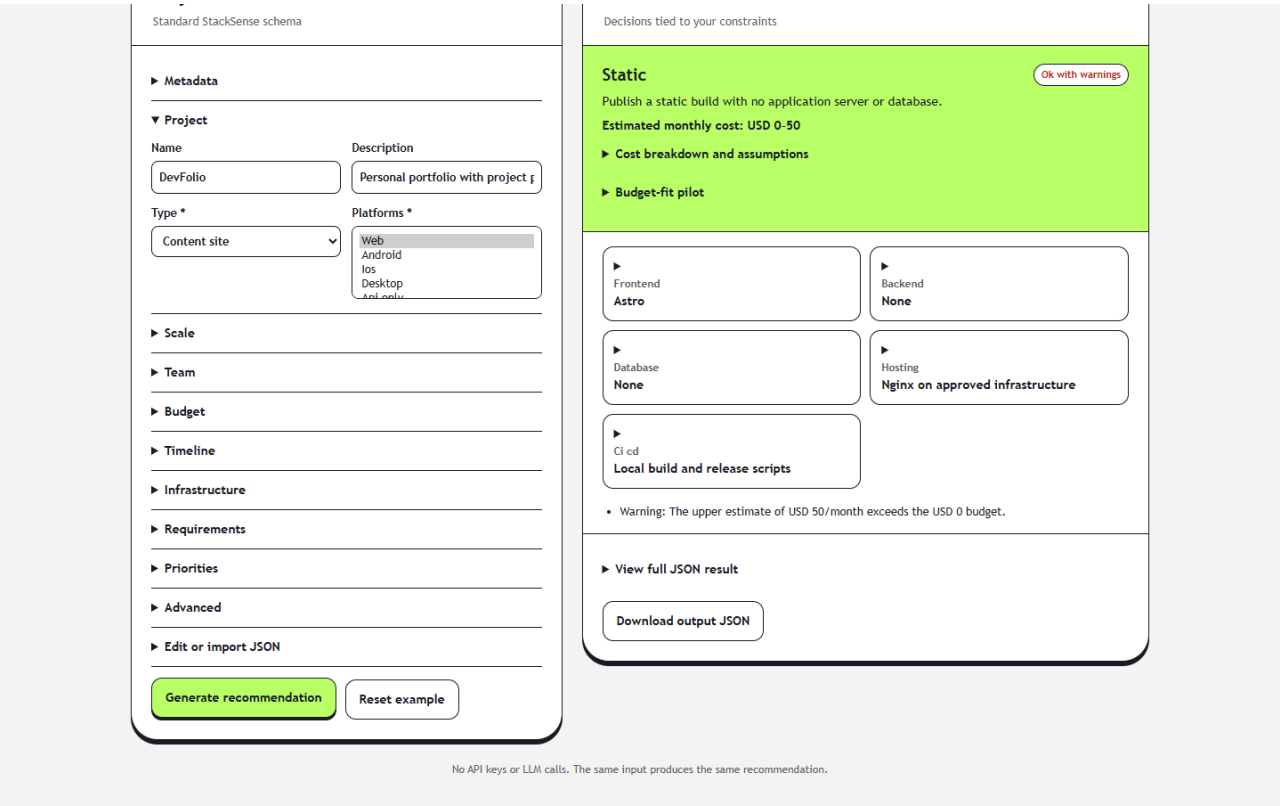


7. Captured edge cases

Zero budget with a positive modeled minimum

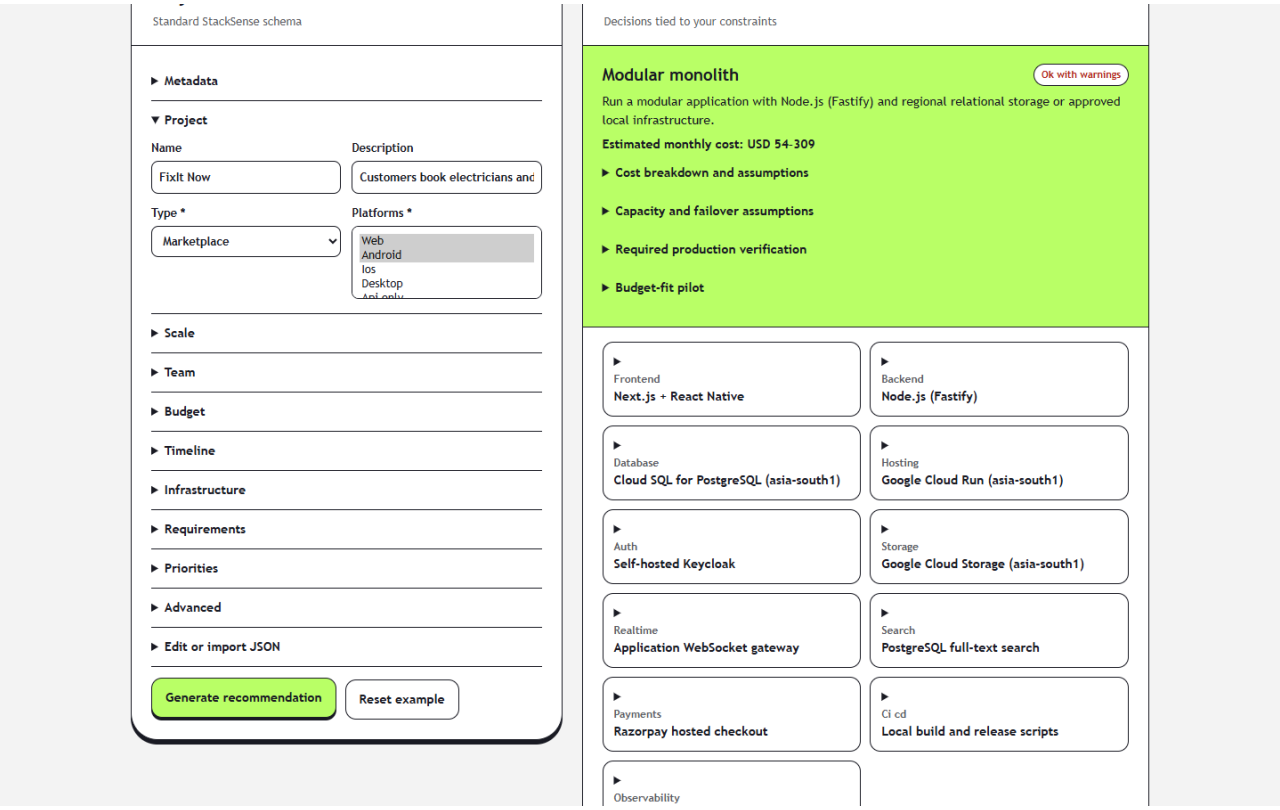


Air-gapped portfolio with local serving



7. Captured edge cases

GCP override with an Indian region



8. Validation

The Node suite checks all six public outputs for determinism and reconciles cost line items with totals. Mutations cover explicit provider selection, cloud storage consistency, VM mode, air-gap isolation, budget rejection, unsupported residency, platform coverage, malformed input and cost-priority sensitivity. Chrome ran the public and selected edge cases through file import, verified results against the Node engine and tested JSON download. Screenshots are from those runs.

9. Limitations and future work

The catalog and ratings are curated rules. Throughput and cost require calibration with regional quotes and workload benchmarks. Unsupported residency is treated conservatively. Legal eligibility, identity contracts, payment processing, licenses, and third-party retention require manual review. Availability and latency are targets to test. Legacy system migration, tenant isolation design and end-of-life detection need deeper profiles.

10. References and third-party tools

StackSense Challenge problem statement and public scenario JSON, supplied with Synergy '26.

AWS Fargate, RDS and S3 pricing: aws.amazon.com/fargate/pricing/; aws.amazon.com/rds/postgresql/pricing/; aws.amazon.com/s3/pricing/.

Azure regional service reference: azure.microsoft.com/en-us/explore/global-infrastructure/products-by-region/.

Cloud Run regions: cloud.google.com/run/docs/locations. Cloud SQL pricing: cloud.google.com/sql/pricing.

PostgreSQL search: postgresql.org/docs/current/textsearch.html. Offline Android: developer.android.com/topic/architecture/data-layer/offline-first.

Keycloak identity: keycloak.org/docs/latest/server_admin/. Recovery: docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/plan-for-disaster-recovery-dr.html.

Runtime: Node.js and browser APIs. Report and capture tooling: ReportLab, Playwright and Chrome. See INPUT-RESEARCH.md and component source fields for additional references.

11. AI assistance declaration

OpenAI Codex assisted with implementation, interface changes, test design, report drafting and evidence capture. The runtime uses deterministic code and makes no model calls.